

Sales Data Analysis & Web Dashboard

Complete Project Guide, End-to-End Workflow, Tech Stack & Interview Script

1. Executive Summary & Project Overview

This project is an **end-to-end data engineering and analytics solution**. It processes raw sales transactions containing real-world data noise (duplicates, missing fields, case inconsistencies, negative values), cleans and engineers features, performs exploratory data analysis (EDA), generates automated reports and charts, and serves an interactive web dashboard on **localhost**.

2. Technology Stack

Category	Technology / Library	Role & Purpose in Project
Language	Python 3.12	Core programming language for data generation, analysis, and dashboard backend.
Data Processing	pandas, numpy	Data manipulation, vector calculations, cleaning nulls/duplicates, and feature engineering.
Data Visualization	matplotlib, Plotly Express	Static chart exporting (PNGs) and interactive dynamic charts for web dashboard.
Web Application	Streamlit	Lightweight Python framework powering the localhost interactive web dashboard.
Version Control	Git, GitHub	Source control management, remote repository tracking, and portfolio hosting.

3. End-to-End Project Workflow

Step 1: Raw Data Generation (generate_data.py)

Creates a realistic dataset of 5,000+ orders with synthetic messiness: stray whitespaces, mixed casing (lowercase/uppercase), null prices/quantities, duplicate OrderIDs, and negative quantities to mirror production database flaws.

Step 2: Data Cleaning Pipeline (analysis.py & app.py)

- Text Standardization: Applied `str.strip().str.title()` to Region, Category, PaymentMethod.
- Deduplication: Removed duplicate OrderID entries.
- Anomaly Correction: Converted invalid negative quantities using `abs()`.
- Missing Value Strategy: Dropped missing key metrics (UnitPrice/Quantity) and imputed missing ages with column median.

Step 3: Feature Engineering

Engineered business-critical columns: $\text{Revenue} = \text{UnitPrice} * \text{Quantity} * (1 - \text{DiscountPct}/100)$, Year, and Month (YYYY-MM).

Step 4: Statistical EDA & Insights Generation

Aggregated total revenue (\$5.82M), total order volume (4,701), average order value (\$1,238.55), top 5 revenue-generating products, category breakdowns, and peak sales months.

Step 5: Visualizations & Reporting

Generated 5 static PNG charts saved into /charts directory and generated a formatted executive summary report sales_report.txt.

Step 6: Interactive Localhost Web Dashboard (app.py)

Built a full Streamlit web app displaying live KPI metric cards, interactive Plotly charts, sidebar date/region/category filters, and an interactive data table with CSV export.

Step 7: Version Control & CI/CD

Maintained codebase version control via Git and pushed the complete project to GitHub repository Mamatak16/sales-data-analysis-project.

4. How to Explain this Project in an Interview

A. The 30-Second Elevator Pitch

"I built an end-to-end Sales Data Analytics solution using Python, pandas, Plotly, and Streamlit. The project mimics a real-world enterprise pipeline: it ingests messy sales transaction data, cleans nulls and duplicate entries, engineers net revenue metrics, exports statistical reports, and serves an interactive web dashboard on localhost where stakeholders can slice and dice revenue by region, category, and date range in real time."

B. STAR Method Breakdown (Situation, Task, Action, Result)

STAR Component	Interview Script Content
Situation	Businesses often struggle to extract actionable insights from raw transactional data due to duplicate records, incomplete values, and static reporting tools.
Task	Design a robust data pipeline to clean messy records, compute business KPIs, generate visualization charts, and provide an interactive dashboard for decision-makers.
Action	1. Developed a pandas data pipeline to handle deduplication, text normalization, and median imputation. 2. Engineered Net Revenue and time features. 3. Designed interactive Plotly charts and built a responsive Streamlit dashboard on localhost.
Result	Processed 5,000+ orders, identified key revenue drivers (\$5.82M total revenue, Toys & Clothing as top categories), and reduced reporting latency by serving real-time filters via a local web app.

C. Technical Q&A; Questions You Can Answer

Q1: How did you handle messy real-world data?

Answer: I built a multi-stage cleaning pipeline in pandas. I trimmed stray whitespace and standardized casing using vectorized string operations. I deduplicated OrderIDs, converted negative quantities to absolute values, dropped rows missing core price data, and filled missing customer ages using median imputation to maintain distribution balance.

Q2: Why did you choose Streamlit over traditional frameworks like Flask/Django?

Answer: Streamlit allows rapid prototyping of data applications purely in Python without needing HTML/CSS/JS boilerplate. It integrates seamlessly with pandas and Plotly, enabling real-time reactive filtering with built-in caching via

@st.cache_data for high performance on localhost.

Q3: What key business findings did your analysis uncover?

Answer: Analysis revealed total revenue of \$5.82M across 4,701 clean orders with an Average Order Value of \$1,238.55. Toys (\$1.21M) and Clothing (\$1.18M) were the highest revenue categories, North was the top region (\$1.52M), and September 2024 was the peak sales month (\$292.7K).

5. Local Execution Commands

Task	Command Line
Install Dependencies	<code>pip install -r requirements.txt</code>
Generate Raw Dataset	<code>python generate_data.py</code>
Run Analysis & Visuals	<code>python analysis.py</code>
Launch Localhost Web App	<code>streamlit run app.py</code>